ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea Magistrale in Digital Transformation Management

Fancy Title

Tesi di laurea in:
BUSINESS INTELLIGENCE

Relatore
Matteo Golfarelli

Candidato
Andrea Giovanardi

II Sessione di Laurea
Anno Accademico 2025-2026

Abstract

Max 2000 characters, strict.

Optional. Max a few lines.

Contents

List of Figures

List of Listings

Chapter 1

Introduction

1.1 Context

During one of the audited weeks of my internship, the operational dashboard flagged a clear gap at the station. The AI summary built on top of it explained that difference with full confidence, but the explanation produced was factually incorrect. It pointed to a structural metric that had little to do with the real cause of the difference, proposed corrective actions and root analyses that no manager could realistically carry out. I noticed the error, that morning, because I was familiar with the station and the metric, but a manager reading the same paragraph on a busy day, or less experienced, could have easily taken that for true and acted on it.

That episode marked my thesis beginning and frames the general problem it addresses: what happens when a Large Language Model (LLM) is placed on top of a BI dashboard and asked to turn numbers into decisions? That is a key question considering the setting of a Delivery Station (DS). It is the final node of the logistic network, the place where packages are sorted in the early morning and loaded onto routes to be delivered to final customers. The station where I worked, DFV2 in Udine, runs on a standard daily rhythm: plan the day, run it, measure what happens, adjust for tomorrow. The manager controls the rhythm through a large set of indicators that are read every day to determine if any correction is needed.

Normally the window to act is narrow so it is important that a manager doesn't get the picture late or even wrong, otherwise the chances to fix it would be lost.

Among all the indicators that we used at the station, one metric acts as the headline figure, a single number that captures how efficiently the station operates. Most of the days this number is close to the planned one, but not exactly on it, and that gap is what a manager has to explore, understand and explain. A process that is anything but trivial. This difference is the combined effect of many separate factors: some of them are internal to the station while the rest of them are completely external, and working out which factor and its weight is a task that, before this project, was done completely by hand.

When a station misses its target, the designated manager has to produce a written justification, internally referred to as the "bridge", based on a weekly Excel report that states how the previous one had closed. A manual and slow activity that said nothing about how the current week was going while there was still time to act on it.

This is the exact scenario where a LLM could be the right fit. A model capable of reading the numbers and turning them into paragraphs within seconds, replacing hours of manual assembly. The downside is that the same flexibility that makes it so powerful and useful is also the source of risks. A language model hallucinates on numbers, quietly recomputing a value or inventing one that looks plausible, and it explains causes with the same fluency whether the cause is real or not. For a tool that is used to feed decisions on the floor, those errors are not acceptable, and they must be reduced as much as possible, while keeping in mind that hallucinations cannot be avoided at 100%. A manager must be able to safely rely on the output of the model while at the same time, having the underlying numbers within reach.

1.2 Criticalities

The main difficulty of this project is the grounding problem, understood in a stricter sense than the one commonly used in the literature. It can be decomposed into two main requirements that need to hold together. The first is numerical:

the number must correspond exactly to the value behind the dashboard, not an approximation of it. The second one is causal: every explanation and every recommendation must come from a well-defined set of causes that the domain admits, not from everything that seems reasonable. Figure 1.1 shows the two requirements and the related failure points.

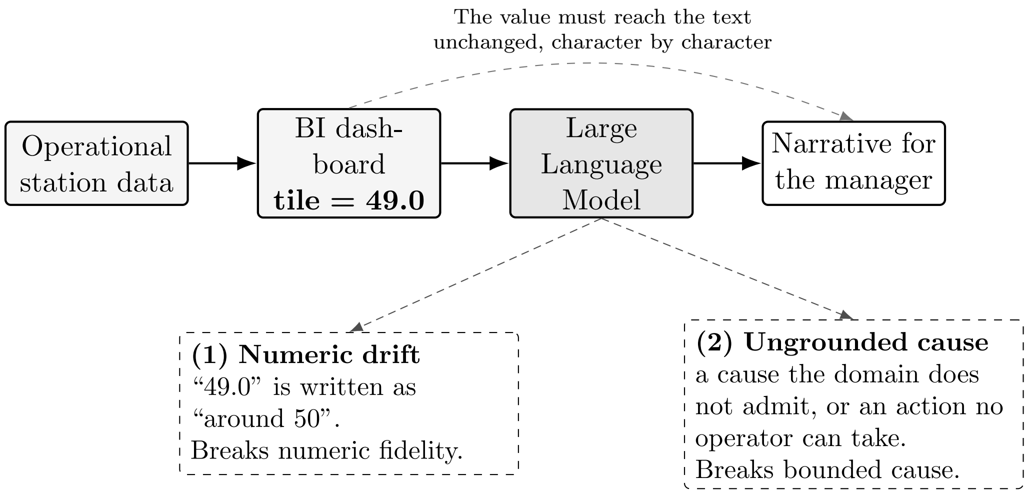

Figure 1.1: From dashboard value to narrative, and the two points where grounding breaks: a number that drifts (1) and a cause the domain does not admit (2).

Consider the numerical side first. The one that is shown to the user must be equal to the retrieved one. If that value is close but not exact, a decision taken on the basis of it can still be wrong. This is more demanding than the usual faithfulness a retrieval system offers, where a claim counts as grounded if it is broadly supported by some documentation. For Operational BI a broad support is not enough.

The causal side is even harder. Reporting a number without changing its value is a matter of staying true to the base data. Explaining why we have that precise value, instead, is connected to the domain knowledge. The set of plausible-sounding causes for any gap is enormous, while the possible set of causes admitted by the domain is small and specific. A narrative defined as "useful" has to stay inside this small set. It should behave like an expert analyst that among the possible explanations of the gap, chooses only the feasible ones, and consequently the actions that a manager can adopt in order to improve the situation. Domain constraint without numerical fidelity buys very little, a right explanation based on a wrong input is meaningless.

Those two requirements matter even more because a wrong output does not stay only on the screen. A manager uses it as the basis for decisions that influence the whole operation. The standard here is therefore demanding, above all on the numbers: a value must be right every time, because one confident mistake, acted upon, can undo a week of planning. The causal side is a different story: the most I can require is that the explanation stays inside what the domain admits. In particular, this is where most of the failures I observed were located: a wrong number is worse, but wrong causes happen more often.

1.3 Framework

The two criticalities highlighted in the previous section shape the questions that guide the thesis. I framed the work around one main question and two subsequent ones, each at a different level.

The first question is empirical and comes before thinking about any solution: what are the common patterns of grounding failure that occur when a language model generates a narrative over a high-density BI dashboard? I wanted a clear picture of how the model fails derived from the field directly, rather than borrowed from the literature. This decision was motivated by the intention to build a design tailored to the real failures.

From the answer to this primary question two design questions follow. The second asks what architecture can be implemented to constrain the faithfulness of the model towards both the numerical and causal sides at once, rather than patching each slip in isolation. The last question is narrower: what role can a curated knowledge tree, a structured map of causes and feasible actions, play as the base structure that bounds what the model is allowed to say?

Those three questions mark the trace that next chapters will follow. The primary is answered by a catalogue of failure modes derived from the internship itself, the second and third by the framework that has been built to mitigate those errors and tested against the same catalogue.

Structure of the Thesis The remainder of this document is organized as follows:

- **Chapter 2** reviews the state of the art and related work, exploring current paradigms for LLM integration, evaluating hallucination metrics, and positioning the work with respect to current methodologies.
- **Chapter 3** introduces the case study, detailing the operational setting, the core metric, the pre-AI baseline, and the single-case methodology adopted during the internship.
- **Chapter 4** presents the proposed solution, detailing the conceptual innovation of the three-contract framework (tile-as-source, curated knowledge tree, and sanitizer) and the system architecture.
- **Chapter 5** focuses on the results and evaluation, analyzing the catalog of grounding failures and providing both a quantitative evaluation on real operational data and real-world qualitative evidence.
- **Chapter 6** discusses the findings, highlighting the thesis contributions and generalizability, acknowledging limitations, and outlining future research directions.

Chapter 2

State of the Art and Related Works

Chapter 1 framed the problem as two requirements that have to hold together. Every number that appears in the generated text must correspond to the actual value of it and every explanation must come from a well-defined domain. This chapter focuses on the existing literature through that double lens. For each family of methods, I run the same two checks: does it guarantee the truthfulness of the number, and does it bound the cause? The map this builds is what the last section uses to present the work.

2.1 The grounding problem

When we talk about grounding, in terms of natural language processing, we usually mean that the output is supported by the source. Retrieval-Augmented Generation (RAG) made this idea popular: a claim counts as grounded when retrieved evidence backs it [1]. There are two main failure modes as stated by the survey on hallucination in text generation [2]. Intrinsic hallucination is text that contradicts the source, and extrinsic hallucination is text that adds something the source cannot confirm. The same literature goes further by separating faithfulness as being coherent with the input given, from factuality intended as the truth in the world

[3]. A text can be faithful but not factual if the input is wrong, but the narrative reports it coherently. At the same time, a text can be factual but not faithful if it states something right in the world, but is not coherent with the input. The requirement that matters for BI is only the first one.

Operational BI requires a higher standard. Generic support is not enough. A figure of 49.0 is either reproduced exactly or it is wrong. Any representation that deviates from the defined value is incorrect. As anticipated in the introduction: a narrative is grounded when every quantitative token is traceable, character by character, to a structured field, and every comparative statement agrees in sign with the source. This aligns more with faithfulness than with factuality, and it is tighter than both because it asks for fidelity to an exact value and not for consistency with a passage. Controlled table-to-text work, the ToTTo dataset for instance [4], pushes the generation in that direction, by asking the model to describe only the given cells without any deviation.

The causal half has no real equivalent in the standard taxonomy. A phrase can have all the right numbers, but still name a cause that does not exist or propose an impossible solution. The standard question when talking about hallucinations is the following: is the claim supported? Rather than: does this cause belong to the domain-defined set? The second question is the dividing line between BI and open-domain generation, and in fact it is the reason why a single definition of grounding is not enough.

2.2 Retrieval-Augmented Generation (RAG)

RAG conditions the model by retrieving passages from a corpus of documents and concatenating them as context into the model's input [1]. It reduces extrinsic hallucinations, since now the model has something to lean on without inventing. It has become a common way to keep a generator close to a body of documents. The evaluation framework follows the same logic: RAGAS scores faithfulness, defined as the degree to which a claim is entailed by the retrieved context [5]. Does the generated phrase follow from the passages? If yes, it is considered grounded.

Here the unit of grounding is the passage, and the test is semantic support. For a textual response this is reasonable, but not for a number. A passage that mentions a figure "in the region of fifty" entails a sentence that says "around fifty", but that sentence fails the operational test I set above. Retrieval works by similarity, not by schema identity. RAG has no concept of canonical identity, so it cannot distinguish between an authoritative value and one that resembles it. RAG is necessary, but not sufficient, since it has no guarantee about the value.

2.3 Prompt engineering and reasoning

A lot of research focuses on the prompt to improve the output, shifting the attention to how something is asked rather than the model itself. Chain-of-thought prompting asks the model to reason by decomposing the work into steps [6]. Self-consistency creates many reasoning paths and keeps the most voted one [7]. Both methods lift the average quality of what comes out, not the single case.

None of them enforces anything. They simply make the correct answer more likely, but they do not make the wrong one impossible. As reported later, even a capable model with a curated but generic prompt tends to invent numbers or causes. Good instructions alone cannot close this gap, but why does it lie to us? When sending a message to an LLM we are making a request, not a command, and since the model itself is a probabilistic generator, it is free to follow our directions or not. On the causal side, prompt engineering helps: it makes the model reason in a cleaner way, and produce fewer off-topic causes. On the operational side it cannot supply the hard guarantee that this use needs, since well-done prompt engineering only pushes the odds; no amount of it can make the value certain.

2.4 Tool use and program-aided generation

A different family gives the model access to external tools. Reasoning + Acting (ReAct) represents an iterative cycle composed of thought, action and observation. It introduces the possibility of performing a search call [8]. Toolformer is a similar approach, capable of deciding on its own when to call an Application Programming Interface (API) [9]. Program-Aided Language (PAL) models take a step forward by generating the code that automatically performs an operation, instead of guessing it [10]. Consequently, the model avoids the hallucination risk associated with computation. In particular, this last idea is close to the one I adopted: taking all the numbers out of the model's hands.

All those methods share one limit. They do a great job at securing the data, meaning the input will likely be correct, but they let the model build on this information without any control or boundary. Fetching the right value is only half of the job; the other half involves representing it correctly. Nothing in these techniques checks that the number that comes out in the sentence is the same as the one that went into the prompt. The tool fixes the input. The output is still free, and a free output can still drift.

2.5 Constrained and guided generation

When we talk about constrained decoding, we refer to a family of methods that force the output into a defined shape independently of the content. Language Model Query Language (LMQL) treats the prompt not as plain text but as a small program characterized by a set of constraints [11]. A typical obligation can be: the response must be shorter than twenty tokens, and the system code will enforce that. Parsing Incrementally for Constrained Auto-Regressive Decoding (PICARD) brings this idea to Structured Query Language (SQL). It is an approach that, by parsing incrementally at each generation step, rejects any token that would make the query invalid, based on the applied constraints. The final result is a query that is syntactically valid, because any token breaking the syntax is rejected and regenerated [12].

Constrained decoding is the closest approach to the sanitizer I implemented and described in Chapter 4. This is motivated by the fact that both of them use code to enforce, before or after the generation, a rule on the output. When talking about PICARD and LMQL, however, they focus only on the form level: a syntactically valid query or well-formed number where it belongs, but they say nothing about the correspondence of the value to the source or the free-text cause belonging to the domain.

2.6 Semantic layers and knowledge graphs

There are two ideas from the data-engineering world that are close to what I need. The first one is the semantic layer, used by tools like data build tool (dbt) and Cube, in which the definition of a metric appears only in a single governed place [13, 14]. The core principle is a single source of truth: the value is computed in one place and read everywhere else, so two reports cannot disagree on the calculation by silently building their own version of the metric. This methodology is the one that I adopt for numbers: take a figure from a single upstream source and copy it, never letting the model work it out. However, that approach is limited because the semantic layer stops before the narrative. It returns clean and canonical numbers, but it doesn't say why that number moved or list, among the possible causes, the one worth reporting.

The second idea regards the causal side. Grounding an LLM in a knowledge graph is one of the most effective ways to keep what the model says inside a fixed domain. How the two can be joined is, by now, well mapped [15]: a graph defines what the domain treats as true and guides the model's reasoning, preventing it from relying on whatever has been absorbed during training. What matters most for my case is the Reasoning over Graphs (RoG). It is an approach that "bounds" every step in the generation process to a specific path in the graph, such that a response can be traced back to a chain of specific relations rather than to a plausible-sounding guess [16]. Other strategies directly feed a smaller portion of the graph into the prompt itself [17], but independently of the methodology that is chosen, the key idea is to give the model a structure and let that structure decide what counts as admissible to say.

Neither family does the whole job alone. The semantic layer returns the right number but no story; knowledge-graph grounding bounds the story, but says nothing about the value. What is needed is a combination of both, together with the downstream check seen above, to make sure that the framework is capable of supporting a correct operational narrative, and it is the one I build in Chapter 4.

2.7 Generative BI and conversational analytics

The applied side of turning dashboards into narrative goes under different names: conversational BI, augmented analytics, generative BI. Its oldest thread is text-to-SQL, where a natural language question is translated into an executable query, scored by benchmarks like Spider that evaluates how accurate the transformation is [18]. A parallel line turns the same question into a chart [19]. Two different outputs originate from the same idea: a table or a picture, nothing more. Any explanation that a manager needs is outside their scope.

It is proven that letting a model handle the numbers is risky [20]. LLMs are unreliable even on simple quantitative reasoning, because of their nature, as stated in Section 2.3. A study about text-to-SQL reveals that, in a production environment, the BI reports present semantic hallucinations with no domain-specific way to track them down [21]. This is the empirical reason behind the strict rule implemented in Chapter 4: the safest number is the one the model never computes.

Over time, commercial tools have moved closer to the narrative side. Tableau Pulse [22] and Copilot in Power BI [23] are capable of writing short insights over tiles, and they are the industry baseline for this work. The most advanced of them documents an approach close to the one that I adopt: Tableau Pulse computes its metric findings with a statistical engine and uses the LLM only to phrase them [24, 25]. Templates and in-context examples reduce the risk, but there is no deterministic check on the printed value, and none of those tools publishes a reproducible measurement of how that grounding holds up practically.

The academic field has grown fast on this topic, during the last year. Some systems are capable of detecting the domain on their own without any human direction and elaborating multi-perspective narratives without any predefined schema or concepts [26]. Others, especially in finance, join causal-discovery statistical algorithms with budgetary variance to automate root cause diagnosis [27]. This is the case closest to mine in terms of goal, since it also tries to explain a gap.

Another approach works on the same structure of data as I do. Paradigms like Conversational and LLM-assisted Online Analytical Processing (OLAP) are capable of bringing natural language to the multidimensional cube. Normally a query is used to retrieve the information. Now the system is not limited to the conversion of text to a query, but also describes what comes back. One such system is Vocalizing OLAP (VOOL): a modular insight-based framework that extracts selected quantitative insights and turns them into a description [28]. It does not use every insight: VOOL picks only the relevant ones and only those become part of the narration. In the field of multidimensional cubes, this last approach is the closest point to a real narrative over structured analytical data. It stays accurate by building descriptions from statistics and templates, without interpreting the results. The numbers are safe, but the reason behind them is left to the reader. Then the wall again. What happens when an LLM is put inside the loop to carry out the analysis? It can hallucinate on numbers. One of the most recent attempts, a system for LLM-assisted navigation, keeps the language model far from figures. The model does only two things: it ranks the candidate views, and it explains why. A separate OLAP engine then computes every value in order to preserve number fidelity [29].

These are the nearest neighbors of the present work, and they differ from it in two aspects: their causes are generated freely or discovered statistically, not bound to any curated knowledge tree, and they implement numerical fidelity by keeping the numbers away from the model, not by a deterministic contract over the generated narrative.

2.8 Evaluating hallucination and faithfulness

The way faithfulness is measured decides what is counted as solved. A critical part, in fact, is choosing the right approach to the scoring, one that avoids marking something as "addressed" that is not. FActSCore is the most fine-grained approach: it works by breaking a long text into smaller fragments called "atomic facts" which are single elementary claims, and by calculating the percentage of those that are supported by the source [30]. HaluEval [31] and TruthfulQA [32], instead, build benchmarks of hallucinated or misleading content. A benchmark is a fixed and shared dataset of test cases that lets different systems be compared on the same ground. SelfCheckGPT goes for a different approach: with no need for any external source, it exploits the model itself by running the same request several times. The fundamental idea behind this sampling-based check is: the model is asked for the same thing repeatedly, if the answer is fabricated, it drifts since the LLM will invent slightly different versions each time; if the multiple answers agree with each other it is considered a real fact [33].

All those instruments were not built for operational BI. For example, an atomic-fact check has no notion of the exact value, while the sampling-based detector is probabilistic: the result is not reproducible since a run is not deterministic. A retrieval faithfulness score measures support, not identity [5]. And more to the point, none of them distinguishes between a numerical error and a causal one, which is the very distinction this work is built around. For that reason, Chapter 5 measures the presence rate of each failure mode, compared across all the configurations, so the effectiveness of each contract can be read directly.

2.9 Positioning of this work

Together, all the families above show a pattern. Each of them secures at most one of the two requirements: numerical or causal. Prompting improves how the model reasons but, while it reduces domain disconnections, it is a request not a command. Tool use works well for grounding numbers, but the narrative is left free. The part related to guided decoding is stricter on form, but it does not mention the correctness of the value or the cause. The semantic layer hands the official number and stops there; knowledge-graphs bound the explanation, but say nothing about the real value.

The applied systems capable of building a narrative fall into three groups. Tableau

Pulse is capable of writing insights over tiles; it describes the fidelity mechanism without any measure of it, and on the causal side it surfaces statistical drivers without binding them to a domain. A newer group of academic systems produces a real day-after narrative: one of these approaches is capable of detecting the domain on its own, while another can identify the root cause by performing a statistical analysis, but in both the numbers generated by the model are not guaranteed. OLAP-narrative systems like VOOL, instead, focus on the number's fidelity by keeping the LLM away from them, yet they stop at description or navigation, and none of them ties the cause to a curated set. With all of them the day-after explanation, in the end, rests on trust and not on a check.

Table 2.1 summarizes all those approaches. The two middle columns contain the requirements from Chapter 1, and the last one asks whether the approach produces a written justification. In particular, in the last row a yes appears in both requirements, but the two carry different meanings. The numeric one is hard because every value is a copy of the upstream one and a deterministic check guarantees that. The causal one is soft: a knowledge tree defines the domain, but there is no enforcement of that as it happens on the value's side.

Approach	**Exact number**	**Bounded cause**	**Day-after**	
RAG	no	no	no	
Prompt engineering	no	no	no	
Tool use	partial	no	no	
Constrained decoding	no	no	no	*techniques*
Semantic layers	yes	no	no	
Knowledge graphs	no	partial	no	
Text-to-SQL / NL2VIS	partial	no	no	
Commercial BI assistants	partial	partial	yes	
LLM-generated narratives	no	partial	yes	*narrative*
LLM-assisted OLAP	yes	no	partial	
This thesis	**yes** (hard)	**yes** (soft)	**yes**	

Table 2.1: Existing approaches against the two requirements of operational BI narrative, and whether they produce a day-after explanation at all.

Figure 2.1 represents the same idea on the two requirements alone. Numerical fidelity is located on the x-axis, causal grounding on the y-axis, and each of them has three possible states: none, partial, full. This work sits in the top-right corner, while every other family lands off a side.

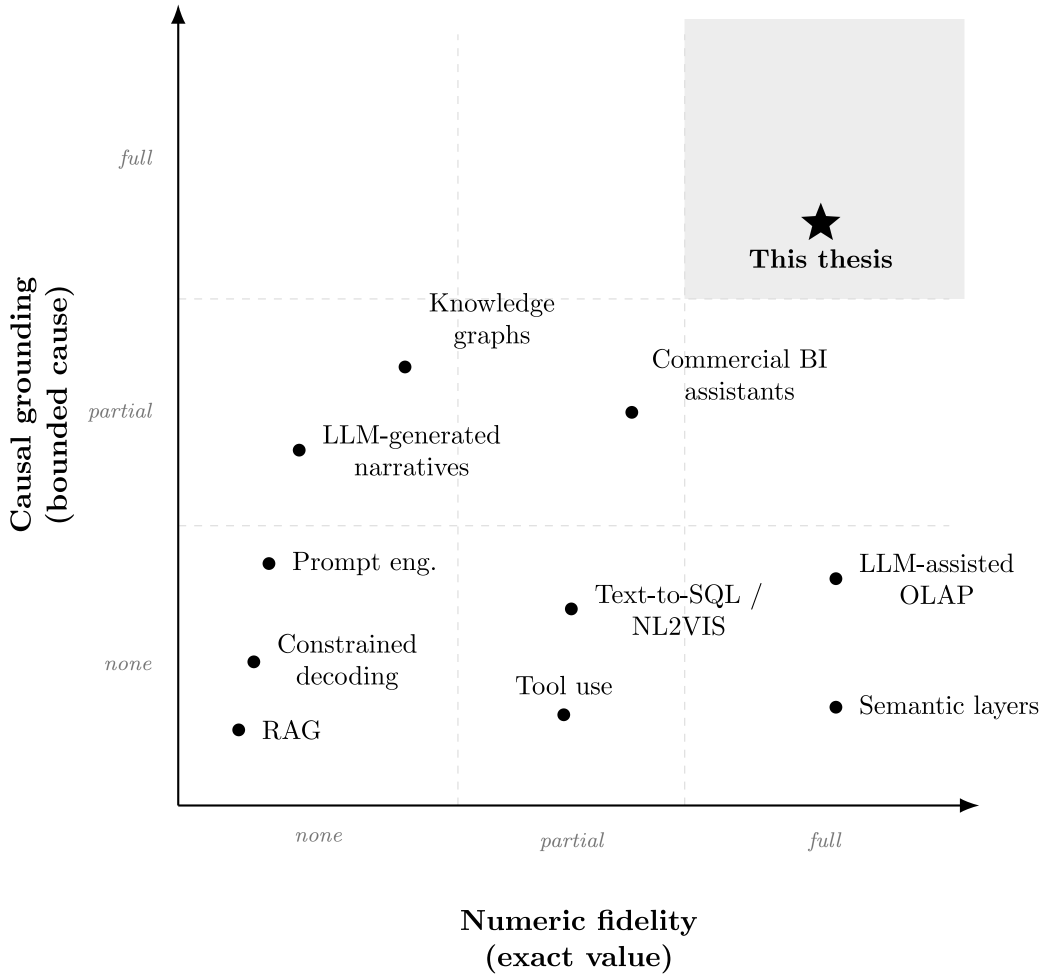

Figure 2.1: The reviewed approaches on the two requirements of operational BI: numeric fidelity (horizontal) and causal grounding (vertical).

As presented, across the ten families reviewed, not one fully addresses both requirements while also writing the explanation a manager reads the day after. There is a reason for it: the two sides are not the same kind of problem. The number can be made a hard guarantee, while the cause only bounded to what the domain admits. The chapters that follow go in that direction, exploring the case where the gap first appeared.

Chapter 3

The industrial case study

This chapter covers the foundations of my case study. It is about the operational setting, the reference indicator the daily review turns on, and what builds it up. Furthermore, it describes all the manual practices done before any automated support and the related weaknesses.

3.1 Operational setting

The setting is the one introduced in Chapter 1: I carried out my internship at DFV2, a last-mile delivery station located in Udine. The focus of my work is not the station as a whole, but a specific weekly task carried out by managers across Europe. Each week a manager has to read the gap and understand what stands behind it, in order to explain it. In my case, this explanation does not stay local: it goes up to a higher team that oversees the whole country and looks at every DS that missed its target.

3.2 The main indicator and its decomposition

The headline indicator can be defined as output quantity over resources used to produce it, in simple words a ratio. The difference between what was planned and the realised value is the gap that my project focuses on. This indicator is then decomposed into a pre-defined and fixed set of categories that are mutually exclusive. The set is organized on two levels: top-level classes that can be decomposed into the sub-factors that compose them (Figure 3.1), this gives depth without losing the structure.

The categories do not contribute to the total gap as a raw value, but they get scaled by its own weight, which is heterogeneous and change across all the class. This means that a large movement in a lightly-weighted value can matter less than a smaller movement in a heavy one. There is also a residual term that absorbs whatever difference the named categories do not explain. It guarantees that the parts add back to the whole, and prevents anything from being forced into a named category where it does not belong.

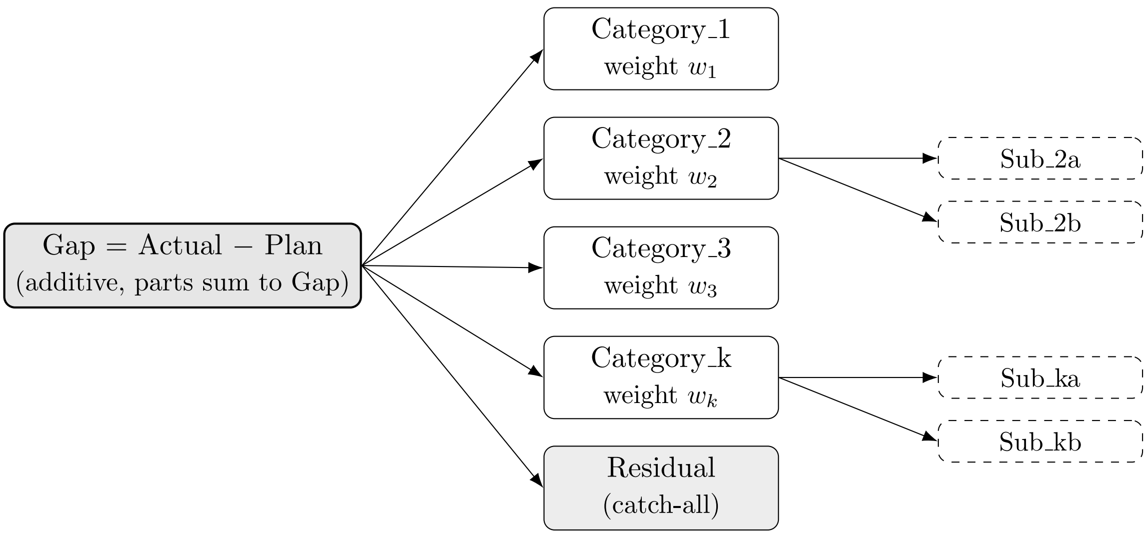

Figure 3.1: Two-level additive decomposition of the gap (anonymised). Top-level categories are mutually exclusive; selected categories open into sub-factors; a residual term closes the sum.

Table 3.2 makes clear why the gap must be decomposed: the categories offset each other, some of them pushes up the difference while others down. A category that helps can hide a problematic one, so the net figure on its own is a poor guide on where the problems are located. Before my project, this whole reading was done manually, which is the starting point of the next section.

Component	**Raw movement**	**Weight** w_i	**Contribution** (= mov. × w_i)
Category_1	+1.50	0.30	+0.45
Category_2	−2.00	0.40	−0.80
of which Sub_2a			−0.55
of which Sub_2b			−0.25
Category_3	−1.20	0.50	−0.60
Category_k	+1.00	0.10	+0.10
of which Sub_ka			+0.18
of which Sub_kb			−0.08
Residual			−0.35
Total (Gap)			**−1.20**

Table 3.1: Illustrative numeric example of the gap decomposition with synthetic values and weights.

3.3 Pre-AI Baseline

The manual workflow for monitoring (weekly cadence, tile-by-tile extraction) and for producing the bridge document (subjective cause selection, unconstrained ranking, experience-dependent actions).

3.4 Three Weaknesses

Limited reactivity, subjective root-cause selection, and analyst dependence, mapped explicitly to the three contracts proposed in Chapter 4.

3.5 Methodology

Single-case longitudinal design where the researcher is embedded in the operational team for six months. Development followed an iterative pattern: observe a failure in the AI output, isolate it, fix it, validate against real data, check for regressions. Failure instances were collected and classified into families for the analysis in Chapter 5. A synthetic dataset preserving the structural properties of the real data built for the controlled evaluation.

Chapter 4

Proposed Solution: Three-Contract Framework

4.1 Core Concept

Introduction of the three contracts (tile-as-source, curated knowledge tree, and post-generation sanitizer) as a unified solution to the grounding problem, each acting at a different point of the generation chain.

4.2 Prompt Engineering

The design of the system prompt as an explicit encoding of fidelity rules, with one directive per observed failure mode rather than generic accuracy instructions.

4.3 Architecture

The overall system architecture of the AI-BI pipeline, from data aggregation to final narrative output.

```java
public class HelloWorld {
  public static void main(String[] args) {
    // Prints "Hello, World" to the terminal window.
    System.out.println("Hello, World");
  }
}
```

4.4 Application

How the framework was derived from the iterative development of the case study, showing the concrete implementation of each contract.

You may also put some code snippet (which is NOT float by default), eg: section 4.4.

4.5 Fancy formulas here

Chapter 5

Results and Evaluation

5.1 Failure Analysis

A catalog of the main grounding failure modes observed and an explanation of how the proposed framework mitigates them.

5.2 Quantitative Assessment

A controlled evaluation using mock data to compare different system configurations.

5.3 Qualitative Assessment

Two to three real-world qualitative episodes drawn from the case study to provide practical, operational evidence of success.

Chapter 6

Discussion and Future Directions

6.1 Synthesis

How the observed results successfully answer the initial research question.

6.2 Contributions

The specific value added to the existing literature regarding LLMs and BI.

6.3 Generalizability

An assessment of how well the three-contract framework can be adapted and applied to other contexts or industries.

6.4 Limitations

A transparent look at the limits of the current thesis work.

6.5 Future Developments

Potential next steps, such as extending the framework to support interactive Q&A functionalities based on the same underlying schema.

Bibliography

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Curran Associates, Inc., 2020.

[2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[3] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[4] Ankur P. Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. Totto: A controlled table-to-text generation dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1173–1186. Association for Computational Linguistics, 2020.

[5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for*

Computational Linguistics (EACL): System Demonstrations, pages 150–158. Association for Computational Linguistics, 2024.

[6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837. Curran Associates, Inc., 2022.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates, Inc., 2023.

[10] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023.

[11] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023.

[12] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models.

In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9895–9901. Association for Computational Linguistics, 2021.

[13] dbt Labs. dbt semantic layer and metricflow. `https://docs.getdbt.com/docs/build/about-metricflow`, 2024.

[14] Cube Dev. Cube: the semantic layer for data apps. `https://cube.dev/`, 2024.

[15] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[16] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

[17] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In *Proceedings of the First Workshop on Matching From Unstructured and Structured Data (MATCHING 2023)*, pages 70–98. Association for Computational Linguistics, 2023.

[18] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921. Association for Computational Linguistics, 2018.

[19] Leixian Shen, Enya Shen, Yuyu Luo, Xiaocong Yang, Xuming Hu, Xiongshuai Zhang, Zhiwei Tai, and Jianmin Wang. Towards natural language interfaces for data visualization: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 29(6):3121–3144, 2023.

[20] Yizhang Zhu, Shiyin Du, Boyan Li, Yuyu Luo, and Nan Tang. Are large language models good statisticians? In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Curran Associates, Inc., 2024.

[21] Jeho Choi. Fact-consistency evaluation of text-to-sql generation for business intelligence using exaone 3.5. *arXiv preprint arXiv:2505.00060*, 2025.

[22] Salesforce Tableau. Tableau pulse: Ai-generated metric insights. `https://www.tableau.com/products/tableau-pulse`, 2024.

[23] Microsoft. Copilot in power bi. `https://learn.microsoft.com/power-bi/create-reports/copilot-introduction`, 2024.

[24] Salesforce Tableau. The insights platform and insight types in Tableau Pulse. `https://help.tableau.com/current/online/en-us/pulse_insights_platform_insight_types.htm`, 2024.

[25] Salesforce AI Research. Developing reliable LLM-powered insight summarization for Tableau Pulse. `https://www.salesforce.com/blog/tableau-pulse/`, 2024.

[26] Ran Zhang, Mohannad Elhamod, et al. Data-to-dashboard: Multi-agent llm framework for insightful visualization in enterprise analytics. *arXiv preprint arXiv:2505.23695*, 2025.

[27] Hejing Chen, Yixuan Lu, Yijing Wei, Jinyan Lyu, Ruibo Wu, and Chen Chen. Causal-llm: A hybrid framework for automated budgetary variance diagnosis and reasoning. In *Proceedings of the 2026 International Conference on Big Data and Internet of Things and Cloud Networks (BDICN)*. Association for Computing Machinery, 2026.

[28] Matteo Francia, Enrico Gallinucci, Matteo Golfarelli, and Stefano Rizzi. Vool: A modular insight-based framework for vocalizing olap sessions. *Information Systems*, 129:102496, 2025.

[29] Xiufeng Liu and Yanyan Yang. Llm-assisted conversational navigation over multidimensional data cubes. *Expert Systems with Applications*, 328:132958, 2026.

[30] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100. Association for Computational Linguistics, 2023.

[31] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. Halueval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6449–6464. Association for Computational Linguistics, 2023.

[32] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3214–3252. Association for Computational Linguistics, 2022.

[33] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9004–9017. Association for Computational Linguistics, 2023.

Acknowledgements

Optional. Max 1 page.